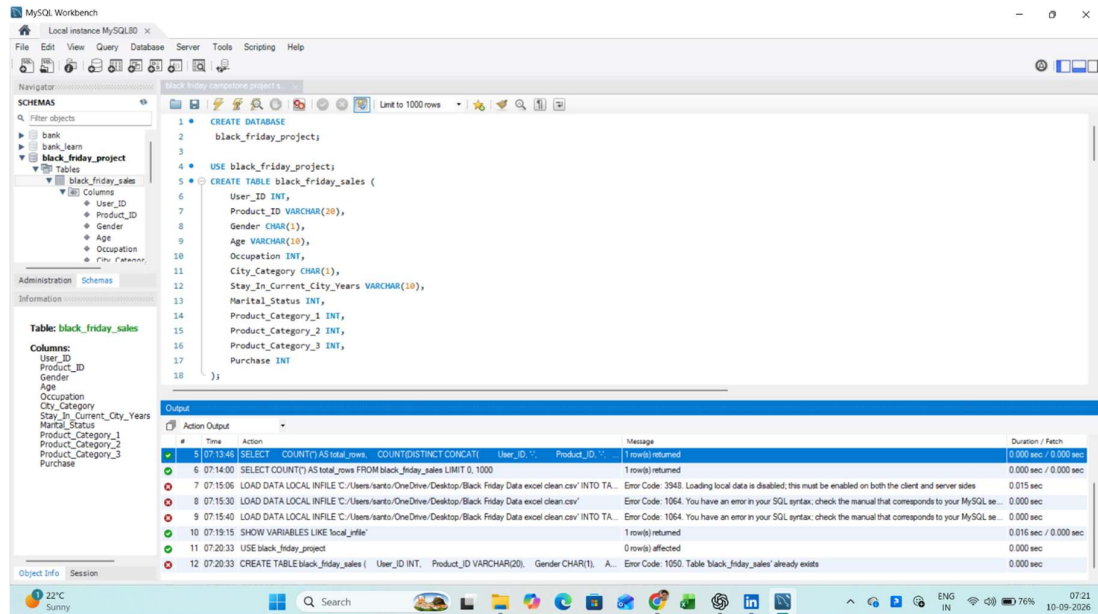


BLACK FRIDAY SALES ANALYSIS

SQL Analysis & Queries

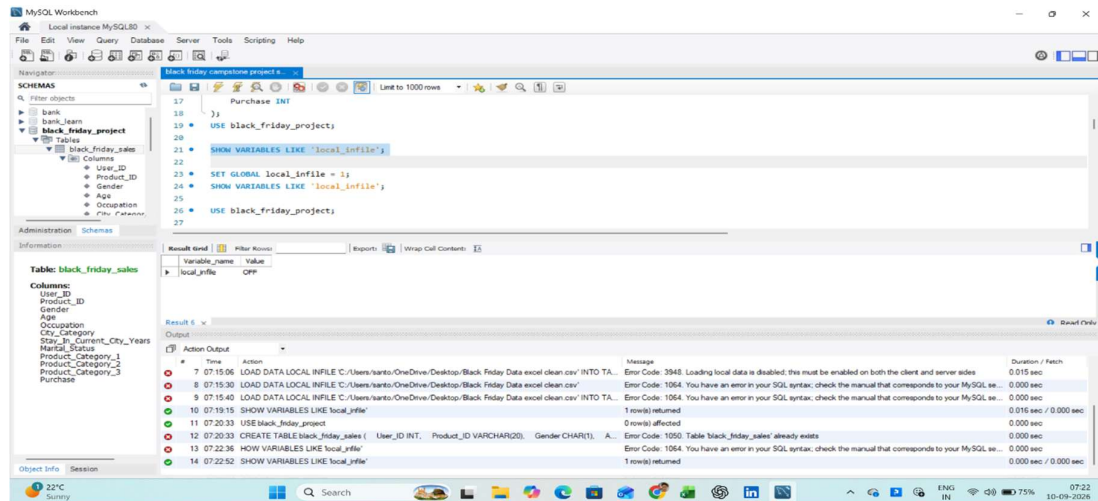
Data Analytics with GenAI Capstone Project

1. Database and Table Creation

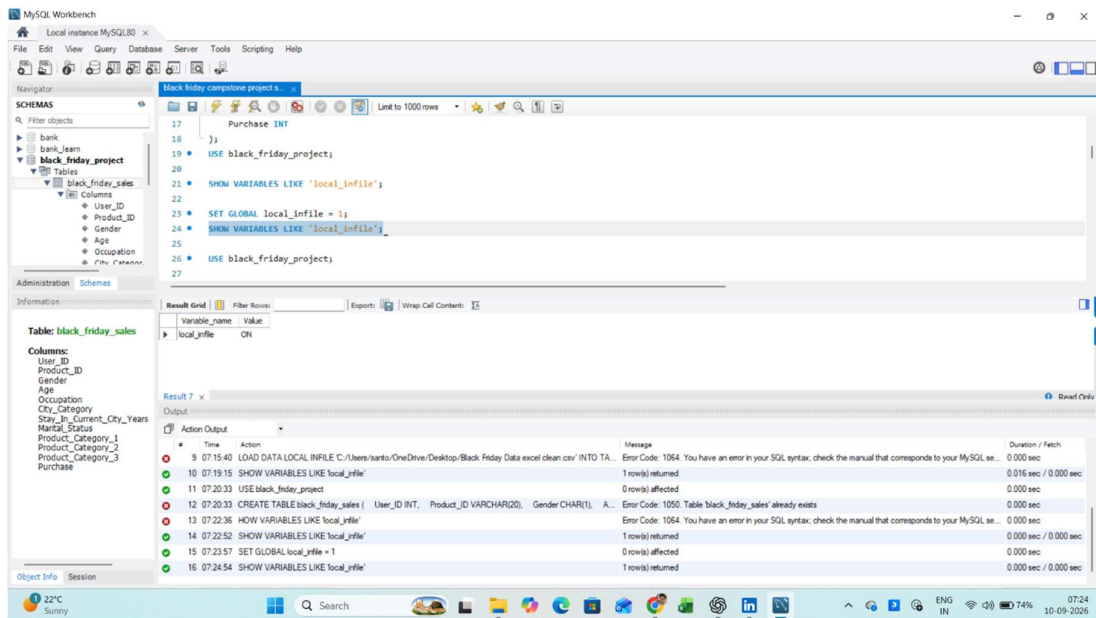


2. Checking Local Data Loading Configuration

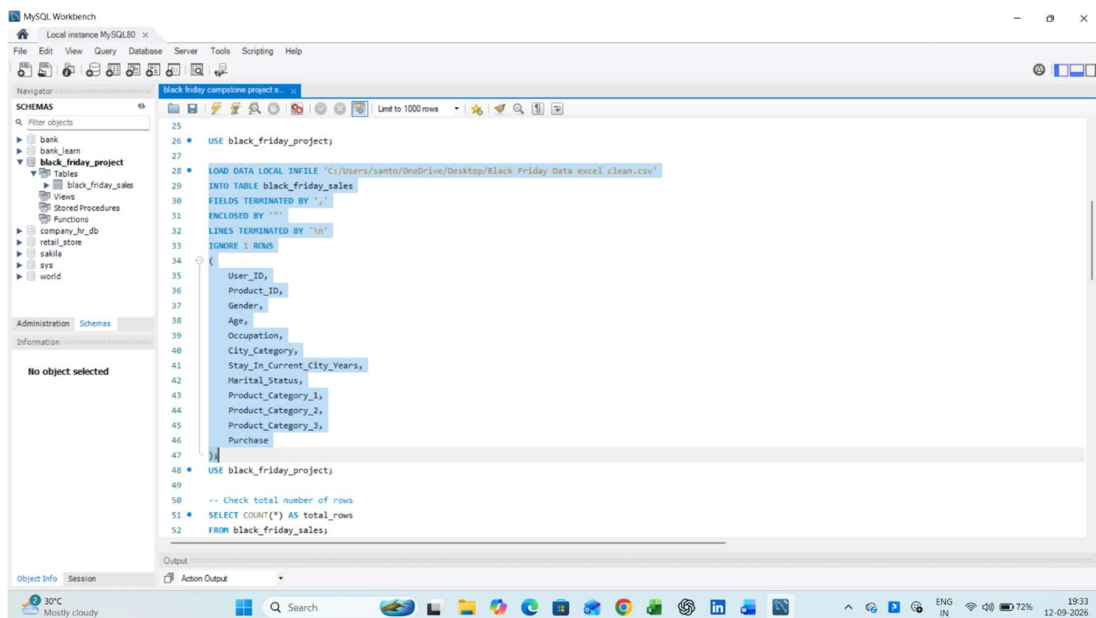
A) Local_infile was initially off

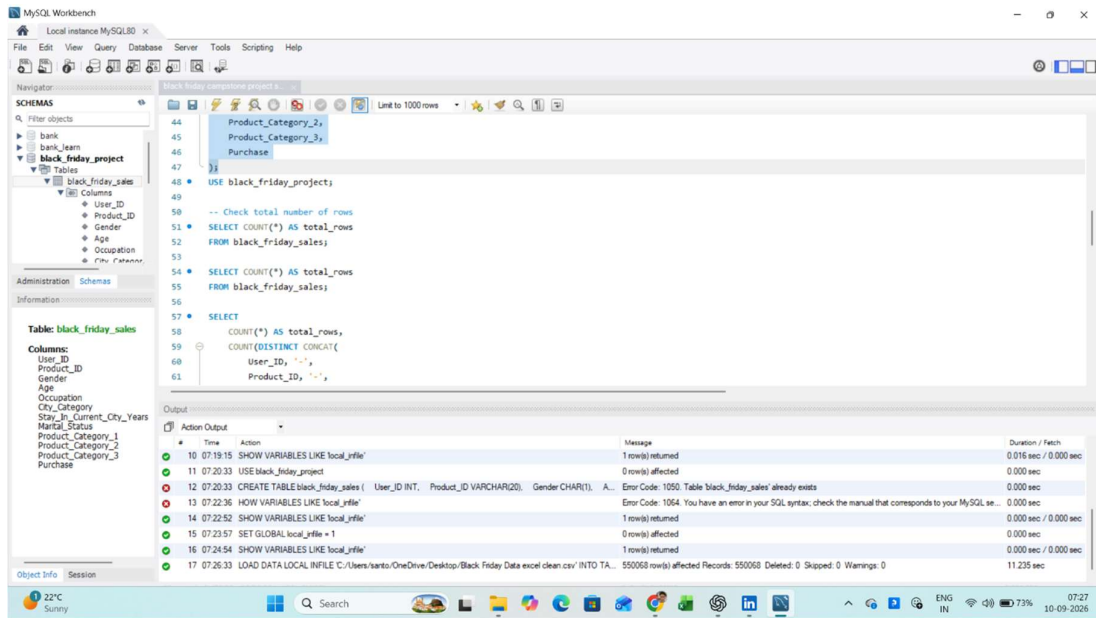


B) Enabling Local Data Loading in MySQL

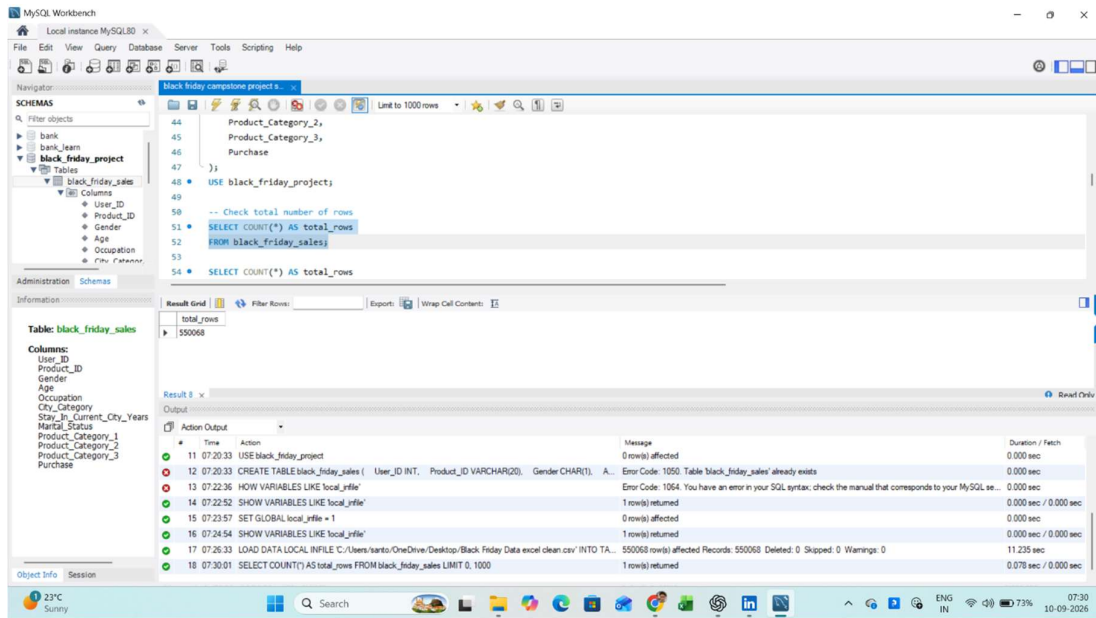


C) IMPORTING DATA FROM LOCAL SOURCE





5. Successful Data Import and Row Count Verification



7. Checking for Duplicate Records

The screenshot shows the MySQL Workbench interface. The left sidebar displays the database schema, including the `black_friday_sales` table with columns: `User_ID`, `Product_ID`, `Gender`, `Age`, `Occupation`, `City_Category`, `Stay_In_Current_City_Years`, `Marital_Status`, `Product_Category_1`, `Product_Category_2`, `Product_Category_3`, and `Purchase`. The main editor shows a SQL query:

```
SELECT
COUNT(*) AS total_rows,
COUNT(DISTINCT CONCAT(
User_ID, '-',
Product_ID, '-',
Gender, '-',
Age, '-',
Occupation, '-',
City_Category, '-',
Stay_In_Current_City_Years, '-'))
```

The bottom panel shows the execution results. The `Result Grid` displays two columns: `total_rows` and `distinct_rows`, both with a value of 550068. The `Action Output` panel shows the execution log, including the query execution time and the number of rows affected.

The screenshot shows the MySQL Workbench interface. The left sidebar displays the database schema, including the `black_friday_sales` table with columns: `User_ID`, `Product_ID`, `Gender`, `Age`, `Occupation`, `City_Category`, `Stay_In_Current_City_Years`, `Marital_Status`, `Product_Category_1`, `Product_Category_2`, `Product_Category_3`, and `Purchase`. The main editor shows a SQL query:

```
SELECT
COUNT(*) AS total_rows,
COUNT(DISTINCT CONCAT(
User_ID, '-',
Product_ID, '-',
Gender, '-',
Age, '-',
Occupation, '-',
City_Category, '-',
Stay_In_Current_City_Years, '-'))
FROM black_friday_sales
```

The bottom panel shows the execution results. The `Result Grid` displays two columns: `total_rows` and `distinct_rows`, both with a value of 550068. The `Action Output` panel shows the execution log, including the query execution time and the number of rows affected.

9. Checking for NULL Values Across All Columns

The screenshot shows the MySQL Workbench interface. The left sidebar displays the database schema, including the 'black_friday_sales' table. The main editor contains a SQL query that checks for NULL values across all columns of the 'black_friday_sales' table. The query is as follows:

```
SELECT
  SUP(User_ID IS NULL) AS null_user_id,
  SUP(Product_ID IS NULL) AS null_product_id,
  SUP(Gender IS NULL) AS null_gender,
  SUP(Age IS NULL) AS null_age,
  SUP(Occupation IS NULL) AS null_occupation,
  SUP(City_Category IS NULL) AS null_city_category,
  SUP(Stay_In_Current_City_Years IS NULL) AS null_stay_years,
  SUP(Marital_Status IS NULL) AS null_marital_status,
  SUP(Product_Category_1 IS NULL) AS null_product_category_1,
  SUP(Product_Category_2 IS NULL) AS null_product_category_2,
  SUP(Product_Category_3 IS NULL) AS null_product_category_3,
  SUP(Purchase IS NULL) AS null_purchase
FROM black_friday_sales;

DESCRIBE black_friday_sales;
```

The result grid shows the output of the query, with columns for each NULL check and the table description. The table description shows the following columns:

Field	Type	Null	Key	Default	Extra
Occupation	int	YES			
City_Category	char(10)	YES			
Stay_In_Current_City_Years	varchar(10)	YES			
Marital_Status	int	YES			
Product_Category_1	int	YES			

10. Validating Data Types and Basic Statistics

The screenshot shows the MySQL Workbench interface. The left sidebar displays the database schema, including the 'black_friday_sales' table. The main editor contains a SQL query that validates data types and basic statistics for the 'black_friday_sales' table. The query is as follows:

```
SUP(Purchase IS NULL) AS null_purchase
FROM black_friday_sales;

DESCRIBE black_friday_sales;

SELECT
  COUNT(*) AS total_transactions,
  COUNT(DISTINCT User_ID) AS unique_customers,
  COUNT(DISTINCT Product_ID) AS unique_products,
  SUP(Purchase) AS total_sales,
  SUM(SUM(Purchase)) AS average_purchase;
```

The result grid shows the output of the query, with columns for each NULL check, the table description, and the basic statistics. The table description shows the following columns:

Field	Type	Null	Key	Default	Extra
Occupation	int	YES			
City_Category	char(10)	YES			
Stay_In_Current_City_Years	varchar(10)	YES			
Marital_Status	int	YES			
Product_Category_1	int	YES			

The Action Output shows the execution of the query, with the following results:

#	Time	Action	Message	Duration / Fetch
16	07:24:54	SHOW VARIABLES LIKE local_infile	1 row(s) returned	0.000 sec / 0.000 sec
17	07:25:33	LOAD DATA LOCAL INFILE 'C:/Users/lenovo/OneDrive/Desktop/Black Friday Data excel clean.csv' INTO T...	550068 row(s) affected Records: 550068 Deleted: 0 Skipped: 0 Warnings: 0	11.235 sec
18	07:30:01	SELECT COUNTRY AS total_rows FROM black_friday_sales LIMIT 0, 1000	1 row(s) returned	0.076 sec / 0.000 sec
19	07:32:01	SELECT COUNTRY AS total_rows FROM black_friday_sales LIMIT 0, 1000	1 row(s) returned	0.076 sec / 0.000 sec
20	07:32:37	SELECT COUNTRY AS total_rows, COUNT(DISTINCT CONCAT(User_ID, Product_ID)) AS unique_products	Err: Code: 1064 You have an error in your SQL syntax; check the manual that corresponds to your MySQL s...	0.000 sec
21	07:32:54	SELECT COUNTRY AS total_rows, COUNT(DISTINCT CONCAT(User_ID, Product_ID)) AS unique_products	1 row(s) returned	11.953 sec / 0.000 sec
22	07:36:29	SELECT SUM(SUM(Purchase)) AS average_purchase	1 row(s) returned	2.734 sec / 0.000 sec
23	07:38:36	DESCRIBE black_friday_sales	12 row(s) returned	0.000 sec / 0.000 sec

SQL EXPLORATORY AND BUSINESS ANALYSIS

The screenshot displays the MySQL Workbench interface with a SQL query and its results. The query is as follows:

```
SELECT
  COUNT(*) AS total_transactions,
  COUNT(DISTINCT User_ID) AS unique_customers,
  COUNT(DISTINCT Product_ID) AS unique_products,
  SUM(Purchase) AS total_sales,
  ROUND(AVG(Purchase), 2) AS average_purchase,
  MIN(Purchase) AS minimum_purchase,
  MAX(Purchase) AS maximum_purchase
FROM black_friday_sales;
```

The results are shown in a table with the following columns: total_transactions, unique_customers, unique_products, total_sales, average_purchase, minimum_purchase, and maximum_purchase. The values are: 550068, 5891, 3631, 5095812242, 9263.97, 12, and 23961 respectively.

The Action Output pane shows the execution of the query, including the time taken and the number of rows returned. The query was executed successfully, returning 1 row.

The Object Info pane shows the table structure for black_friday_sales, including columns: User_ID, Product_ID, Gender, Age, Occupation, City_Category, Stay_In_Current_City_Years, Marital_Status, Product_Category_1, Product_Category_2, Product_Category_3, and Purchase.